



Optimism in Equality Saturation

RUSSEL ARBORE, University of California, Berkeley, USA

ALVIN CHEUNG, University of California, Berkeley, USA

MAX WILLSEY, University of California, Berkeley, USA

Equality saturation is a program optimization technique based on non-destructive rewriting and a form of abstract interpretation called e-class analysis. Existing e-class analyses are pessimistic and therefore typically imprecise when analyzing cyclic programs, such as those in SSA form. We show that a straightforward optimistic variant of e-class analysis can result in unsoundness, due to a subtlety in how e-graphs represent programs. We propose an abstract interpretation algorithm that circumvents this issue and can optimistically analyze e-graphs during equality saturation. This results in a unified algorithm for optimistic analysis and non-destructive rewriting. We implement a prototype abstract interpreter and equality saturation tool for SSA programs. Our tool exhibits precision improvements over pure abstract interpretation (without rewriting) and pessimistic e-class analysis on example programs. Additionally, its performance is comparable to existing abstract interpretation and e-class analysis techniques.

CCS Concepts: • **Theory of computation** → **Program analysis**; **Equational logic and rewriting**.

Additional Key Words and Phrases: Equality Saturation, Static Single Assignment (SSA), Abstract Interpretation.

ACM Reference Format:

Russel Arbore, Alvin Cheung, and Max Willsey. 2026. Optimism in Equality Saturation. *Proc. ACM Program. Lang.* 10, PLDI, Article 224 (June 2026), 24 pages. <https://doi.org/10.1145/3808302>

1 Introduction

Optimizing compilers transform an input program into a “better” program. Most compilers implement a transformation-based approach—an input program is modified by a sequence of separate passes. In this regime, the order of the individual passes matters, because each pass destroys the previously known program (this is often referred to as the phase ordering problem).

Equality Saturation. Equality saturation is an alternate approach to performing compilation, where transformations are implemented as term rewrites and a data structure called an e-graph keeps track of all intermediate programs and known equivalences between them [51]. This approach enables *non-destructive* rewriting, which bypasses the phase ordering problem. Recent implementations of equality saturation also include *e-class analysis*, which is a form of abstract interpretation [13, 56]. Rewrites and analyses cooperate: rewrites can be conditioned on analysis facts, and equalities from rewriting combine analysis facts into more precise ones [13]. These systems have been applied in many areas, including floating point accuracy [40], circuit synthesis [14], tensor and linear algebra [54, 57], 3D CAD [36], and imperative program compilation [18, 51].

Authors' Contact Information: Russel Arbore, russel@berkeley.edu, University of California, Berkeley, Berkeley, California, USA; Alvin Cheung, akcheung@cs.berkeley.edu, University of California, Berkeley, Berkeley, California, USA; Max Willsey, mwillsey@berkeley.edu, University of California, Berkeley, Berkeley, California, USA.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/6-ART224

<https://doi.org/10.1145/3808302>

Optimism. Many interesting programs contain loops, which often correspond to cycles in data flow. A strict subset of program analyses, called “optimistic” analyses in the compilers literature, are capable of precisely analyzing cyclic program representations, as opposed to “pessimistic” analyses, which use bottom-up reasoning and therefore struggle to reason about loops. Optimistic analyses, as their name suggests, optimistically assume a potentially unsound analysis fact about program fragments in a cycle and then later refine the analysis [7, 52].

Optimism in Equality Saturation. Incorporating optimistic analyses into equality saturation has not been achieved previously. Existing e-class analyses are pessimistic, meaning cyclic programs are not precisely analyzed. Additionally, performing optimistic analyses on e-graphs after rewriting is fraught; *we show that the straightforward approach leads to unsoundness*. Additionally, as optimistic analyses are not incrementally sound, optimistic analyses cannot be interleaved with rewriting in equality saturation, as rewrites are not necessarily revocable in e-graphs. Prior work in equality saturation has identified a specific need for optimism: de-duplicating isomorphic cycles in an e-graph [51, 62, 63]. A technique from traditional compilers, optimistic global value numbering, would solve this problem [1, 7, 44], but is not applicable due to the aforementioned difficulties.

We incorporate optimistic analyses into an equality saturation system for the first time. We describe a prototype combined equality saturation engine and abstract interpreter for imperative programs, based on a Static Single Assignment (SSA) program representation with novel semantics. Our prototype analyzes example programs more precisely than standard abstract interpretation, pessimistic e-class analysis, or compilation with gcc or clang. It also presents a solution to the cycle de-duplication problem identified in prior work. In summary, our contributions are as follows:

- We identify the core issue preventing optimism in equality saturation—equality saturation creates ill-formed represented graphs which poison optimistic analyses (Section 3).
- We propose a SSA form program representation that 1) can be easily embedded into an e-graph and 2) has a simple semantics amenable to abstract interpretation (Section 4).
- We describe optimistic analyses over SSA programs embedded in e-graphs—discovered equalities make analyses more precise, but also create ill-formed represented graphs. We propose an abstract interpretation algorithm that computes an abstraction that is both sound and optimistic. We prove that this algorithm always computes a sound analysis of the well-formed represented graphs in an e-graph and study its complexity (Section 5).
- We build a prototype program optimizer in Rust that combines equality saturation and optimistic analyses and evaluate it on example programs requiring both rewriting and optimistic analysis to fully optimize; existing techniques cannot fully optimize these examples, while our tool can. We also study the cost of our algorithm on randomly generated programs, compared against standard abstract interpretation and e-class analysis. (Sections 6 and 7).

2 Background

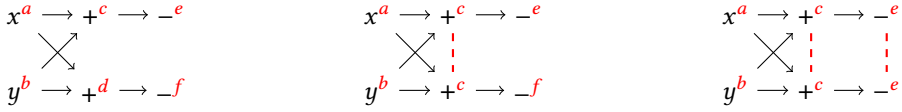
2.1 Equality Saturation

Equality saturation [51] performs non-destructive rewriting over the e-graph data structure [37]. Several implementations exist, the two most popular being egg [56] and egglog [60].

2.1.1 E-Graphs. An *e-graph* stores an equivalence (sometimes congruence) relation of terms.

Definition 2.1 (E-Graphs). An e-graph is pair of finite sets of *e-nodes* $\mathcal{N}$ and *e-classes* $\mathcal{C}$ where:

- An e-node $n \in \mathcal{N}$ is a *function symbol* f of arity k and a tuple of e-classes $(i_1, \dots, i_k) \in \mathcal{C}^k$. An e-node “is” a function f when f is its symbol. The j th input class is written as n_j .
- There is a surjection $[\cdot] \in \mathcal{N} \rightarrow \mathcal{C}$ that defines e-class membership. $[\cdot]^{-1} \in \mathcal{C} \rightarrow \mathcal{P}(\mathcal{N})$ gives the pre-image of an e-class under $[\cdot]$.



(a) Initial e-graph representing $-(x + y)$ and $-(y + x)$. (b) E-Graph after applying the rewrite $x + y \Rightarrow y + x$. (c) E-Graph after rebuilding.

Fig. 1. Example e-graphs during equality saturation. Symbols are e-nodes, solid edges connect e-nodes to arbitrary e-nodes in their input e-classes, and dashed red edges connect e-nodes in the same e-class, following the convention from [51]. We super-script e-nodes with an identifier (also in red) for their e-class. Note that we connect e-nodes $\leftarrow$ input e-classes, indicating the flow of data, as in [8].

Prior work defines "represented terms" to describe programs existing in an e-graph [56]. We instead define *represented graphs*, since we will use e-graphs to represent cyclic terms in this paper.

Definition 2.2 (Cyclic Terms). A cyclic term is a pair of finite sets of nodes $\mathcal{V}$ and ids $\mathcal{I}$ where:

- A node $v \in \mathcal{V}$ is a function symbol f of arity k and a tuple of ids $(i_1, \dots, i_k) \in \mathcal{I}^k$. A node "is" a function f when f is its symbol.
- There is a bijection $v \in \mathcal{I} \rightarrow \mathcal{V}$.

For brevity, we conflate $\mathcal{V}$ and $\mathcal{I}$ (and simply refer to $\mathcal{V}$)—for example, we write the j th input to a node n as n_j and say $n_j \in \mathcal{V}$ (while technically we must apply v to map from the input id to a node). The graph may contain cycles. We often call cyclic terms just "graphs".

Definition 2.3 (Represented Graphs). A cyclic term $\mathcal{V}$ is a represented graph of an e-graph $(\mathcal{N}, \mathcal{C})$ when there is a map $m \in \mathcal{V} \rightarrow \mathcal{N}$ such that for all nodes $v \in \mathcal{V}$, v and $m(v)$ have the same function symbol, $\forall i \in \mathbb{Z}, 1 \leq i \leq k \implies m(v)_i = [m(v_i)]$, where k is the arity of v 's function symbol, and $[\cdot] \circ m$ is a surjection. In other words, m is a homomorphism from the cyclic term into the e-graph preserving dependency structure up to equivalence and all e-classes are mapped into¹.

Figure 1 shows three example e-graphs. The e-nodes and e-classes of an e-graph are mutually recursive—e-nodes are members of e-classes and e-nodes have e-classes as inputs rather than e-nodes, since it doesn't matter which e-node in an e-class is being referred to (they're all equivalent).

2.1.2 Batched Rewriting and Rebuilding. A typical equality saturation workflow has three steps:

- (1) The e-graph is seeded with an initial graph (the graph we are interested in optimizing).
- (2) Rewrites rules are repeatedly applied to the e-graph. This may reach a fixpoint (called saturation), but is not guaranteed to—applications often stop rewriting after a timeout [50].
 - (a) Rewrites query the e-graph using the *e-matching* procedure to find terms represented by the e-graph matching the left hand side of the rewrite, like the $x + 0$ in $x + 0 \Rightarrow x$.
 - (b) Matches yield substitutions, which are used to instantiate the right hand side pattern—these new terms are inserted and asserted equal with matched terms.
 - (c) Discovered equalities may imply further equalities by congruence, so *rebuilding* is run to discover these new equalities explicitly.
- (3) *Extraction* picks a graph (usually a tree or DAG) represented by the e-graph that is optimized with respect to some cost model [5, 20, 51, 56, 58]. Extraction is an e-class analysis [56] and prior work has encountered a similar problem to the one we describe in Section 3 [57].

¹Technically, prior definitions of represented terms do not require that all e-classes are represented. We use this requirement to simplify some definitions in Section 5, but represented graphs can be understood without this requirement.

2.1.3 E-Class Analysis. The *e-class analysis* framework adds abstract interpretation capabilities into equality saturation, enabling the safe application of rewrite rules that are conditioned on some analysis result [13, 56]. E-class analysis associates a semi-lattice element with each e-class and propagates facts in a bottom-up fashion. Facts are associated with e-classes, rather than e-nodes, because all e-nodes in an e-class are known to be equal—a sound analysis for any e-node in the e-class must be sound for all of the e-nodes. egglog uses a Datalog-like architecture to associate multiple semi-lattice facts with each e-class, but the fundamental mechanism is the same [60]. Facts derived for e-nodes inside the same e-class can be combined to increase precision—since rewrites create larger e-classes, rewriting and e-class analysis are mutually beneficial [13, 56, 60].

Consider the following example of an interval e-class analysis from Coward et. al. [13]. Let our language be arithmetic expressions, and the analysis domain be the set of intervals over the reals. Consider the following expressions over variables x and y , where $x \in [0, 1]$ and $y \in [1, 2]$.

$$\frac{x - y}{x + y} \in [-2, 0] \quad \frac{2x}{x + y} - 1 \in [-1, 1]$$

Since the variables x and y have known intervals, the intervals for the terms can be derived as $[-2, 0]$ and $[-1, 1]$. If rewriting discovers that the two above expressions are equivalent (they are), the e-classes will be merged, and the fact for the new e-class will be their *intersection* (if two equivalent terms have intervals, they both must lie in both intervals). Thus, the new fact for the merged e-class will be $[-1, 0]$, which is more precise than either of the original intervals.

2.1.4 Cycles in E-Graphs. Rewrites can create cycles when a term is discovered equivalent to one of its sub-terms. In prior work, these cycles correspond to infinite families of acyclic terms. In this work, we embed cyclic terms into the e-graph, which requires extending the notion of representation to graphs (Definition 2.3). However, *this does not mechanically change the e-graph*.

2.2 Abstract Interpretation

Abstract interpretation is a framework for approximating the behavior of programs, usually using lattice structures [10, 11]. “Abstractions” characterize what sets of concrete executions are possible in a particular program [10, 12, 23, 29–33, 43, 45, 46, 53]. Often, abstractions characterize what concrete values a program variable may store at some point during program execution—these are called non-relational abstractions, and include intervals [10] and known-bits [53]. Abstractions can also characterize what concrete values a tuple of program variables may store simultaneously—these are called relational abstractions, and include difference bounds [31], octagons [33, 45], pentagons [30], polyhedra [12, 46], Karr’s domain [23], two variables per inequality [43], and equality [6].

Typically, abstract interpretation is described in terms of a *Galois connection* between a *concrete lattice* Σ and an *abstract lattice* $\Sigma^\#$. Concretization and abstraction functions $\gamma \in \Sigma^\# \rightarrow \Sigma$ and $\alpha \in \Sigma \rightarrow \Sigma^\#$ form a Galois connection when for all $s \in \Sigma$ and $s^\# \in \Sigma^\#$, $s \sqsubseteq \gamma(s^\#) \iff \alpha(s) \sqsubseteq^\# s^\#$. An abstraction $s^\# \in \Sigma^\#$ is a *sound over-approximation* of $s \in \Sigma$ when $s \sqsubseteq \gamma(s^\#)$. We can lift a function $f \in \Sigma \rightarrow \Sigma$ to operate on $\Sigma^\#$: $\alpha \circ f \circ \gamma \in \Sigma^\# \rightarrow \Sigma^\#$. γ and α may be uncomputable—an *abstract transformer* $f^\# \in \Sigma^\# \rightarrow \Sigma^\#$ soundly over-approximates f when for all $s^\# \in \Sigma^\#$, $f(\gamma(s^\#)) \sqsubseteq \gamma(f^\#(s^\#))$. We use the $\#$ superscript notation to refer to the abstract counterpart of a concrete object.

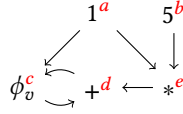
An abstract interpretation can be expressed as a set of equations describing states at program locations as abstract transformers applied to states at predecessor locations. These equations may be mutually dependent. If the abstract transformers are sound, any fixpoint solution to the equations is sound with respect to the program semantics [10]. For some domains, infinite ascending or descending chains may prevent the computation of least or greatest fixpoints, respectively. A sound analysis can be computed by using widening or narrowing, respectively, to compute a *post-fixpoint* instead (abstraction where further iteration of the equations yields a strictly decreased abstraction).

```

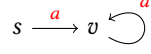
1 let x = 1;
2 while 1 {
3   x = x + (1 * 5);
4 }

```

(a) A simple program with a loop.



(b) DFG of the program.



(c) CFG of the program.

Fig. 2. A program in pseudo-code and as a SSA program. The nodes in the DFG are super-scripted by identifiers in red. The CFG contains two nodes: s , which is the entry point, and v , representing the loop body. ϕ nodes are annotated with CFG vertices. CFG edges are annotated by predicate values that guard control flow.

2.3 Static Single Assignment Form

Static Single Assignment (SSA) form is a category of program representation where 1) every variable has a single definition and 2) every definition is executed before its uses (which is called *dominance*). Variables in SSA form are often instead called *values* to emphasize that they do not change and are unambiguously defined. A special ϕ instruction is used to join data flow at control flow joins [41].

Some SSA form representations use graphs to represent data flow, where nodes represent operators and edges represent dependencies—SSA values are identified by nodes, and ϕ nodes are used to join results at control flow joins. Prior works have multiple names and exact formulations for these graphs. One form is the “sea-of-nodes” graph representing data and control flow [8, 16]. The data flow graph has also been called the “global value graph” (“SSA graphs” are described as combining global value graphs and control flow graphs) [26]. We separate the data flow and control flow portions of a program into a data flow graph (DFG) and a control flow graph (CFG). Figure 2 shows an example program, a corresponding DFG, and a corresponding CFG.

Not all DFGs can be in SSA form; if there is a dependency cycle consisting of non- ϕ operations, there is no order in which the operations can be evaluated (in every serialization of the cycle, at least one definition will not dominate all of its uses). Additionally, it only makes sense to talk about a DFG being in SSA form with respect to some CFG, as the CFG defines dominance among ϕ nodes. We say that a DFG that is in SSA form (with respect to some CFG) is “well-formed”.

3 Challenges in Abstract Interpretation over E-Graphs

We describe in detail why performing abstract interpretation over e-graphs, specifically in the presence of cycles, is challenging. We identify how existing e-class analysis implementations circumvent this issue and argue that this circumvention is insufficient for future use cases.

Recall from Section 2.2 that we can express an analysis of a program as a set of equations over an abstract lattice. We can express an e-class analysis over an e-graph in a similar fashion. Additionally, recall from Section 2.1 that e-class analysis associates an analysis fact with each e-class in an e-graph. Thus, these equations will describe the known analysis fact for each e-class. Each e-node can be abstractly interpreted according to 1) its function symbol and 2) its input e-classes. Additionally, the abstractions of multiple e-nodes in an e-class can be combined via the meet operator to obtain an abstraction of the e-class. If f is the function symbol of an e-node, call $f^\#$ a corresponding abstract transformer. Then, the equation for the abstraction $c^\#$ describing an e-class c is:

$$c^\# = \prod_{n \in [c]^{-1}} f^\#(n_1^\#, \dots, n_k^\#)$$

This formulation is standard from prior e-class analysis literature [13, 56]. Let us now consider applying e-class analysis to a specific example. Figures 3a and 3b show an e-graph and its e-class analysis equations. We use the interval lattice to analyze a very simple e-graph that is representative

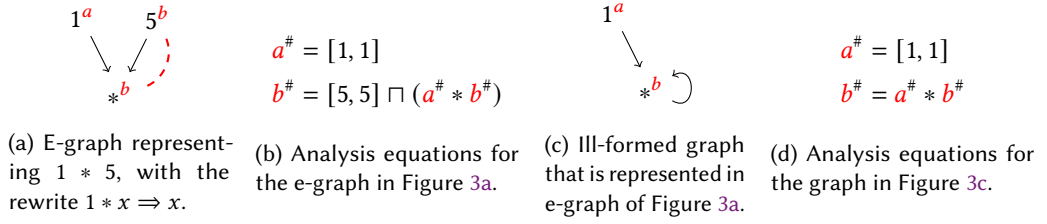


Fig. 3. E-class analysis (with the interval lattice) of a simple e-graph.

of structures seen in existing applications of e-graphs. There are two fixpoints to this set of equations: $a^\# = [1, 1]$, $b^\# = [5, 5]$ (the greatest fixpoint), and $a^\# = [1, 1]$, $b^\# = \perp$ (the least fixpoint). Intuitively, only the first fixpoint makes sense—the second fixpoint implies that there are no possible values for b . This seems false, because the original program, $1 * 5$, clearly evaluates to 5.

The issue is that the rewrite $1 * x \Rightarrow x$ causes the e-graph to represent the graph shown in Figure 3c. The cycle from b to itself prevents a concrete execution from occurring, since the nodes cannot be evaluated in any order that evaluates definitions before their uses. Throughout this paper, we refer to graphs that we cannot assign a semantics to as “ill-formed”. Despite being ill-formed, we can still generate a set of analysis equations for the ill-formed graph based on its structure (shown in Figure 3d). The least fixpoint of this set of equations is $a^\# = [1, 1]$, $b^\# = \perp$ —because an e-class analysis combines analyses for the same e-class with the meet operator, the least fixpoint for the ill-formed graph will “poison” the least fixpoint of the e-class analysis.

This observation seems to run contrary to prior work, where it is assumed that e-class analysis computes a sound abstraction of represented terms in the e-graph [13, 56]. In particular, the represented terms of the e-class b in Figure 3a are $5, 1 * 5, 1 * (1 * 5), \dots$, all of which evaluate to 5. The trick that prior work uses, implicitly, is to compute the *greatest* fixpoint of the e-class analysis equations (or a different (post-)fixpoint whose computation is incrementally sound [13]). This circumvents the “poisoning” caused by the ill-formed graph in Figure 3c, since the greatest fixpoint of the equations in Figure 3d is $a^\# = [1, 1]$, $b^\# = [-\infty, \infty]$, and for all intervals a , $a \sqcap [-\infty, \infty] = a$.

The e-graph shown in Figure 3a is representative of existing use cases, where cycles are only created by rewrite rules that equate a term with one of its sub-terms (for example, rewriting $1 * x$ to x). In these use cases, all programs of interest are acyclic, and thus computing a greatest fixpoint of the e-class analysis is acceptable. However, for future use cases, we want to incorporate optimism, as programs can contain loops or recursion. We return to the example program shown in Figure 2. Figure 4 shows the DFG for this program with the rewrite $1 * x \Rightarrow x$ applied and the e-class analysis equations for the resulting e-graph². These equations have three fixpoints:

- (1) $a^\# = [1, 1]$, $b^\# = \perp$, $c^\# = [1, 1]$, $d^\# = \perp$
- (2) $a^\# = [1, 1]$, $b^\# = [5, 5]$, $c^\# = [1, \infty]$, $d^\# = [6, \infty]$
- (3) $a^\# = [1, 1]$, $b^\# = [5, 5]$, $c^\# = [-\infty, \infty]$, $d^\# = [-\infty, \infty]$

Fixpoint #1 is the least fixpoint and derives $b^\# = d^\# = \perp$, which is unsound for *well-formed* represented graphs. Fixpoint #3 is the greatest fixpoint, so it is sound, but it does not determine that the loop induction variable is always positive. Fixpoint #2 suffers from neither deficiency (it is sound and optimistic). For a suitable choice of abstract domain (restrictions described in Definition 4.5), the primary contribution of this work is an algorithm which computes an analysis that is 1) sound for well-formed represented graphs in an e-graph and 2) optimistic, so programs with loops or

²Unlike the graph in Figure 3, the semantics of a DFG with ϕ nodes depends on a walk through a CFG. We will properly define these semantics in Section 4—for now, this analysis can be understood to be flow-insensitive.

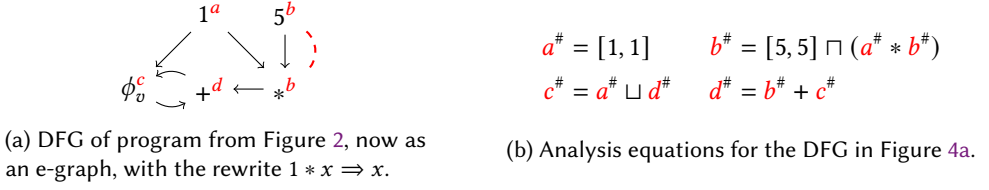


Fig. 4. E-class analysis (flow-insensitive, with the interval lattice) of the program from Figure 2.

recursion can be analyzed more precisely (Section 5). Prior work in e-graph extraction (which is a kind of e-class analysis) tackles a similar problem by filtering cycles [57]. This requires removing e-nodes from the e-graph, which explicitly removes represented graphs. This may harm precision of both rewriting and analysis. Additionally, one cannot remove an e-node from a singleton e-class (otherwise, the e-class would have no e-node to produce during extraction). Our method does not suffer from these issues because it does not modify the e-graph.

4 Semantics and Abstract Interpretation of SSA Programs

We describe a graph-based SSA form representation that is similar to the sea-of-nodes IR [8], but separates data flow and control flow into two graphs (the DFG and the CFG). This separation 1) simplifies specifying the semantics and 2) eases rewriting, as the DFG is easy to treat as an e-graph while the CFG is not. We describe a semantics for this representation, consisting of a denotational component for interpreting the DFG and an operational component for interpreting the CFG. These semantics are similar to prior work describing semantics for the sea-of-nodes [16], with the key difference that the denotational component is parameterized by a particular walk through the CFG, rather than relying on a supplied prior evaluation for ϕ nodes—this is used to justify the correctness of our optimistic e-class analysis algorithm in Section 5.3. We describe how abstract interpretation can be performed on this representation, including flow-insensitive and flow-sensitive variants.

4.1 Structure of SSA Programs

A SSA program is a pair $(\mathcal{S}, \mathcal{G})$ of a DFG and a CFG. A DFG $\mathcal{S}$ is a cyclic term (Definition 2.2) that may contain nodes with ϕ function symbols. A CFG $\mathcal{G} = (V, s, P)$ is a graph with vertices V , a distinguished entry $s \in V$, and a predecessor function $P \in V \rightarrow (V \times \mathcal{S})^*$ which maps each vertex to a tuple of its predecessors—a predecessor is a vertex paired with a node from $\mathcal{S}$, which guards control flow on the edge. We use guard conditions rather than branch instructions, as it makes specifying the semantics simpler. The DFG and CFG are mutually dependent—the CFG uses DFG values as guards, and ϕ DFG nodes depend on control flow to select inputs (specifically, ϕ function symbols are parameterized by a non-entry vertex from the CFG). For simplicity, we assume that vertices have at most two predecessors, that predecessor vertices are unique, and that all vertices v where the ϕ_v function symbol exists in the DFG have exactly two predecessors³.

4.2 Concrete Semantics

The semantics of SSA programs consists of a denotational component for defining the value of each DFG node (in some concrete domain Σ , which we assume is a complete lattice) parameterized by a walk in the CFG, and an operational component for defining what walks are possible. We assume that every non- ϕ function symbol, f , appearing in the DFG can be interpreted as a function $f \in \Sigma^k \rightarrow \Sigma$,

³Any program with an arbitrary number of predecessors per CFG vertex can be transformed into an equivalent program with at most two predecessors per vertex. We assume this both in our formalization and in our implementation.

where k is the arity of f . We also assume that there is a $\mathcal{T} \subseteq \Sigma$ which is the set of “true” values (used to evaluate guards). A walk W in a program with DFG $\mathcal{S}$ and CFG $\mathcal{G} = (V, s, P)$ of length $k \in \mathbb{Z}_{>0}$ is a tuple of vertices $W \in V^k$ such that $W_1 = s$ and $\forall i \in \mathbb{Z}, 1 \leq i < k \implies (\exists g \in \mathcal{S}, (v_i, g) \in P(v_{i+1}))$ —in other words, a walk is a sequence of vertices in $\mathcal{G}$ that starts at s and traverses edges in $\mathcal{G}$. $\mathcal{W}$ is the set of all walks in a program. We often discuss the “predecessor” walk of a walk W , written as W' . We say that $W = W'; v$ when W is the walk W' with the addition of the vertex v at the end.

We first define the denotational semantics of nodes in a DFG $\mathcal{S}$. The denotation of a node is a function from CFG walks to sets of domain values—that is, $\llbracket \cdot \rrbracket \in \mathcal{S} \rightarrow (\mathcal{W} \rightarrow \Sigma)$.

Definition 4.1 (Denotation of DFG Nodes). The denotation of a node $n \in \mathcal{S}$ is defined as:

$$\llbracket n \rrbracket(W) \triangleq \begin{cases} \perp & n \text{ is a } \phi_v \wedge W = (s) \\ \llbracket n \rrbracket(W') & n \text{ is a } \phi_{v'} \wedge W = W'; v \wedge v \neq v' \\ \llbracket n_1 \rrbracket(W') & n \text{ is a } \phi_v \wedge W = W''; p; v \wedge \exists g \in \mathcal{S}, (p, g) = P(v)_1 \\ \llbracket n_2 \rrbracket(W') & n \text{ is a } \phi_v \wedge W = W''; p; v \wedge \exists g \in \mathcal{S}, (p, g) = P(v)_2 \\ f(\llbracket n_1 \rrbracket(W), \dots, \llbracket n_k \rrbracket(W)) & n \text{ is a } f \in \Sigma^k \rightarrow \Sigma \end{cases}$$

Note that this function is only well defined for certain SSA programs. According to the structure described in Section 4.1, a DFG is just a cyclic term—consider the cyclic term shown in Figure 3c. $\llbracket b \rrbracket$ is circularly defined (for any walk in any CFG), and is thus not well defined. To address this issue, we define a subset of SSA programs as *well-formed*.

Definition 4.2 (Well-formed SSA Programs). A SSA program with DFG $\mathcal{S}$ and CFG $\mathcal{G} = (V, s, P)$ is well-formed if and only if:

- (1) Every cycle in $\mathcal{S}$ contains at least one node whose function symbol is a ϕ symbol, and...
- (2) For every node $n \in \mathcal{S}$ where n is a ϕ_v , let $((p_1, g_1), (p_2, g_2)) = P(v)$, then,
 - (a) $\forall W \in \mathcal{W}, p_1 \in W \implies \Phi_{\text{pred}}(n_1) \subseteq W \wedge \Phi_{\text{pred}}(g_1) \subseteq W$, and...
 - (b) $\forall W \in \mathcal{W}, p_2 \in W \implies \Phi_{\text{pred}}(n_2) \subseteq W \wedge \Phi_{\text{pred}}(g_2) \subseteq W$,

where:

$$\Phi_{\text{pred}}(n) \triangleq \begin{cases} \{v\} & n \text{ is a } \phi_v \\ \bigcup_i \Phi_{\text{pred}}(n_i) & \text{otherwise} \end{cases}$$

The first condition enforces that data operations can be evaluated in some order, given values for input ϕ s. The second condition is the classic strictness condition of SSA form [41], which states that every value is defined before it is used along every control flow walk⁴. Given a well-formed SSA program and an arbitrary control flow walk, the semantics of any node is well defined.

Next, we define the operational semantics that characterize control flow in a DFG $\mathcal{S}$ and CFG $\mathcal{G} = (V, s, P)$. The core set that is defined is $\mathcal{W}_P$, which is the set of *possible* control flow walks.

Definition 4.3 ($\rightarrow_{\text{WALK}}$). Given $W; v \in \mathcal{W}$ and $v' \in V$, then $W; v \rightarrow_{\text{WALK}} W; v'$ if and only if $\exists g \in \mathcal{S}, (v, g) \in P(v') \wedge \llbracket g \rrbracket(W; v) \in \mathcal{T}$.

Definition 4.4 ($\rightarrow_{\text{WALK}}^*$ and $\mathcal{W}_P$). $\rightarrow_{\text{WALK}}^*$ is the reflexive and transitive closure of $\rightarrow_{\text{WALK}}$. The set of possible walks is $\mathcal{W}_P = \{W \in \mathcal{W} \mid (s) \rightarrow_{\text{WALK}}^* W\}$.

$\rightarrow_{\text{WALK}}$ evaluates guards on control flow edges to determine if it is possible to step from a vertex to a successor vertex. Starting from the entry point, all intermediate walks are possible walks.

⁴More precisely, for every ϕ_v node at some vertex v , the ϕ nodes that are immediately depended on should be defined on all walks that reach v —this is true when the vertices of the depended ϕ nodes dominate the predecessors of v .

4.3 Abstraction of Concrete Semantics

Next, we describe how SSA programs can be abstractly interpreted. For simplicity, we stick to non-relational abstract domains in this paper, as e-class analysis has mostly been limited to non-relational domains in prior work⁵. An abstraction of the concrete semantics of a SSA program will over-approximate the values nodes may evaluate to.

Definition 4.5 (Abstract Interpretation of SSA Programs). An abstract interpretation over SSA programs is a pair $(\Sigma^\#, \gamma)$, where:

- $\Sigma^\#$ is a complete lattice⁶ with a widening operation ∇ where $\forall s_1^\#, s_2^\# \in \Sigma^\#, s_1^\# \sqcup s_2^\# \sqsubseteq s_1^\# \nabla s_2^\#$.
- $\gamma \in \Sigma^\# \rightarrow (\mathcal{W}_P \rightarrow \Sigma)$ is the concretization function, and is also monotone ($\forall s_1^\#, s_2^\# \in \Sigma^\#, s_1^\# \sqsubseteq s_2^\# \implies \gamma(s_1^\#) \sqsubseteq \gamma(s_2^\#)$) and $\gamma(\top^\#) = \top$.
- $\Sigma^\#$ either has only finite ascending chains or all chains $s_0^\# \in \Sigma^\#, s_j^\# \triangleq s_{j-1}^\# \nabla t_j^\#$ for arbitrary $t_j^\# \in \Sigma^\#$ are eventually non-increasing (there exists $k \in \mathbb{N}$ such that $s_k^\# \sqsubseteq s_{k-1}^\#$).
- For the purpose of computing greatest fixpoints, we assume that all descending chains in $\Sigma^\#$ are finite. This simplifies our definitions and soundness proof in Section 5.3. In practice, narrowing can be used to cut off descending chains while maintaining soundness.

The last two points are necessary to ensure termination. The former is a standard assumption in abstract interpretation [10]. The latter is a slight modification of a standard assumption in e-class analysis [13]. Additionally, we assume that every non- ϕ function symbol $f \in \Sigma^k \rightarrow \Sigma$ has a corresponding abstract transformer $f^\# \in \Sigma^{k\#} \rightarrow \Sigma^\#$. An abstract interpretation is *sound* when:

- $\forall (s_1^\#, \dots, s_k^\#) \in \Sigma^{k\#}, f(\gamma(s_1^\#), \dots, \gamma(s_k^\#)) \sqsubseteq \gamma(f^\#(s_1^\#, \dots, s_k^\#))$ for all non- ϕ function symbols (abstract transformers over-approximate the semantics of concrete functions).
- $\forall S^\# \subseteq \mathcal{P}(\Sigma^\#), \sqcup_{s^\# \in S^\#} \gamma(s^\#) \sqsubseteq \gamma(\sqcup S^\#) \wedge \bigcap_{s^\# \in S^\#} \gamma(s^\#) \sqsubseteq \gamma(\bigcap S^\#)$ (join and meet in the abstract domain over-approximate join and meet in the concrete domain).
- $\forall s_1^\#, s_2^\# \in \Sigma^\#, \gamma(s_1^\#) \sqcup \gamma(s_2^\#) \sqsubseteq \gamma(s_1^\# \nabla s_2^\#)$ (widening in the abstract domain over-approximates join in the concrete domain).

We point out two subtleties. First, $\Sigma^\#$ abstracts $\mathcal{W}_P \rightarrow \Sigma$, not Σ (expanded on in Section 4.4). Second, the co-domain of γ is not the same as the co-domain of $\llbracket \cdot \rrbracket$ ($\mathcal{W}_P \rightarrow \Sigma$ vs. $\mathcal{W} \rightarrow \Sigma$). This reflects that $\llbracket \cdot \rrbracket$ is defined over all walks, while we are only interested in abstracting over possible walks. An abstraction $s^\#$ of a node n is *locally sound* for some $W \in \mathcal{W}_P$ if $\llbracket n \rrbracket(W) \sqsubseteq \gamma(s^\#(W))$. An abstraction $s^\#$ of a node n is sound if and only if it is locally sound for all possible walks.

We can compute a flow-insensitive abstract interpretation on a program by computing a fixpoint of a set of equations—this set of equations contains one equation per node in the DFG.

Definition 4.6 (Equations for Abstract Interpretation of SSA Programs). Fix a program with DFG $\mathcal{S}$ and CFG $\mathcal{G} = (V, s, P)$. For a node $n \in \mathcal{S}$, its abstraction $n^\#$ can be written:

- If n is a ϕ_v node for some $v \in V$, then the equation is $n^\# = n_0^\# \sqcup n_1^\#$.
- If n is a $f \in \Sigma^k \rightarrow \Sigma$ node, then the equation is $n^\# = f^\#(n_1^\#, \dots, n_k^\#)$.

A sound abstraction can be computed by 1) starting from $\top$ and iteratively applying the equations, possibly stopping early (narrowing), or 2) starting from $\perp$ and iteratively applying the equations until a (post-)fixpoint is reached, where dependency cycles among the equations are broken with widening (the equation for a widened ϕ node $n \in \mathcal{S}$ is $n^\# = n^\# \nabla (n_0^\# \sqcup n_1^\#)$). The ϕ nodes whose

⁵egglog supports associating a lattice fact with a tuple of e-classes [60], though we are not aware of an application of egglog that uses this capability to perform relational abstract interpretation. In [13], there is some discussion of “relational domains”, but this refers to relational information revealed by rewrites, rather than a relational e-class analysis.

⁶To distinguish from Σ , we sometimes write the lattice operations of $\Sigma^\#$ as $\perp^\#, \top^\#, \sqsubseteq^\#, \sqcup^\#,$ and $\sqcap^\#$.

equations are modified are called “widening points”, and can be determined as follows: compute a weak topological order (WTO) over the CFG [3] and mark all ϕ nodes whose vertex is a “component head” in the WTO as widening points. The WTO identifies a subset of vertices such that all CFG cycles contain at least one node in this subset (called the component heads). All data flow cycles in a well-formed DFG go through ϕ nodes, and by strictness, those cycles always correspond to cycles in the CFG. Thus, every cycle in the DFG contains a ϕ node that is marked as a widening point.

4.4 Flow Sensitivity

In some program representations, non-relational abstractions are computed at and associated with specific program points. This simplifies computing a flow sensitive analysis, as different abstractions can be stored at different program points. However, in our SSA program representation, every value in the DFG is *global* to the entire function, and the fact that a node may evaluate differently on different walks is lifted into the semantics (this is why the co-domain of $\llbracket \cdot \rrbracket$ is $\mathcal{W} \rightarrow \Sigma$, rather than just Σ). Thus, abstract objects do not over-approximate concrete values, but rather concrete values per walk. There are *at least* three ways we can build the lattice of abstract objects $\Sigma^\#$:

- (1) $\Sigma^\#$ contains abstractions of concrete values and γ returns a constant function ignoring its walk argument. This is often called a *flow insensitive* analysis.
- (2) $\Sigma^\#$ contains maps from control flow vertices to abstractions of concrete values and γ returns a function that takes the *last* vertex in the passed walk and uses that vertex to index the map being concretized. This is often called a *flow sensitive* analysis.
- (3) $\Sigma^\#$ contains maps from (up-to) k -tuples of vertices to abstractions of concrete values and γ returns a function that takes the *last* (up-to) k vertices in the passed walk and uses that tuple to index the map being concretized. This is sometimes called a *k-path sensitive* analysis.

Note that none of these options store an abstraction of concrete values per possible walk⁷, since $\mathcal{W}_P$ may be infinite. In the rest of this paper, we will consider flow insensitive analyses—this is because all prior work in e-class analysis that we are aware of describes flow insensitive analyses [13, 56] and due to a limitation of e-graphs and equality saturation which we expand on in Section 6.3. We emphasize that there is no fundamental reason that the analysis algorithm we propose in Section 5 cannot be used for flow sensitive analyses.

5 Optimistic E-Class Analysis

The DFG in a SSA program is normally a single graph—this can be replaced by an e-graph which represents multiple graphs. As shown in Section 3, this has unfortunate consequences when the e-graph represents ill-formed graphs. We expand more on this challenge and propose an abstract interpretation algorithm that computes a sound abstraction of all *well-formed* represented graphs of the e-graph in a SSA program. We also justify why this is the right goal.

5.1 Representing DFGs with E-Graphs

Recall that a SSA program consists of a DFG $\mathcal{S}$ and a CFG $\mathcal{G}$. We now replace $\mathcal{S}$ with an e-graph $(\mathcal{N}, \mathcal{C})$. Values in the program are no longer identified by specific nodes, but now by equivalence classes of nodes. Unlike a normal DFG, we do not give direct semantics to an e-graph—this is because the e-graph may represent ill-formed graphs that prevent $\llbracket \cdot \rrbracket$ from being well-defined.

Definition 5.1 (E-Graph Soundness). Fix an e-graph $(\mathcal{N}, \mathcal{C})$ and CFG $\mathcal{G}$. Call $\mathbb{V}$ the set of represented graphs of the e-graph that are also well-formed DFGs, with respect to $\mathcal{G}$ —for graph $\mathcal{V} \in \mathbb{V}$, call its representation map $m_{\mathcal{V}} \in \mathcal{V} \rightarrow \mathcal{N}$. We say that the e-graph is sound if and only if:

⁷This is the required lattice to represent a meet-over-paths analysis, which is in general incomputable [22].

- (1) There is a single $\mathcal{W}_P$ such that for all $\mathcal{V} \in \mathbb{V}$, the SSA program consisting of DFG $\mathcal{V}$ and CFG $\mathcal{G}^8$ has possible walks $\mathcal{W}_P$, and...
- (2) For that $\mathcal{W}_P$, $\forall \mathcal{V}_1, \mathcal{V}_2 \in \mathbb{V}, \forall v_1 \in \mathcal{V}_1, v_2 \in \mathcal{V}_2, [m_{\mathcal{V}_1}(v_1)] = [m_{\mathcal{V}_2}(v_2)] \implies \forall W \in \mathcal{W}_P, \llbracket v_1 \rrbracket(W) = \llbracket v_2 \rrbracket(W)$.

In other words, all well-formed represented graphs must agree on the set of possible walks (this defines $\mathcal{W}_P$ for a sound e-graph and accompanying CFG) and all nodes in well-formed represented graphs that map into the same e-class must evaluate to the same value on all possible walks.

Our definition of soundness only concerns equivalence of semantics between nodes of *well-formed* represented graphs. This requirement comes from the use of the semantics of nodes in the represented graphs. As discussed in Section 4.2, the semantics of SSA programs are only well-defined for well-formed programs. It does not make sense to define e-graph soundness based on represented graphs that do not have well-defined semantics (the ill-formed graphs). In the rest of this paper, we will assume that e-graphs are always sound—an e-graph with singleton e-classes is trivially sound if the initial graph is well-formed and we assume that rewrites preserve soundness.

5.2 E-Class Analysis of SSA Programs

E-class analysis associates an abstraction with each e-class in an e-graph. As e-graphs do not have a direct semantics, we use the semantics of well-formed represented graphs to define soundness.

Definition 5.2 (E-Class Analysis Soundness). Fix a sound e-graph $(\mathcal{N}, C)$ and CFG $\mathcal{G}$. Call $\mathbb{V}$ the set of represented graphs of the e-graph that are also well-formed DFGs, with respect to $\mathcal{G}$ —for graph $\mathcal{V} \in \mathbb{V}$, call its representation map $m_{\mathcal{V}} \in \mathcal{V} \rightarrow \mathcal{N}$. $s^\# \in \Sigma^\#$ is a locally sound abstraction for some $W \in \mathcal{W}_P$ of an e-class $c \in C$ if and only if $\forall \mathcal{V} \in \mathbb{V}, \forall n \in \mathcal{V}, [m_{\mathcal{V}}(n)] = c \implies \llbracket n \rrbracket(W) \sqsubseteq \gamma(s^\#)(W)$. In other words, all nodes in well-formed represented graphs that map into the e-class c must be over-approximated by $s^\#$. $s^\#$ is a sound abstraction of c if it is locally sound for all $W \in \mathcal{W}_P$.

As in the definition of soundness of e-graphs, the soundness of an e-class analysis depends only on well-formed represented graphs. Ill-formed represented graphs do not necessarily have a well-defined semantics, so it does not make sense to discuss over-approximating their semantics.

A key property of e-class analysis that is well understood is that sound abstractions of e-nodes in the same e-class can be combined using the lattice meet operator [13, 56].

THEOREM 5.3. Fix a sound e-graph $(\mathcal{N}, C)$ and CFG $\mathcal{G}$ with well-formed represented graphs $\mathbb{V}$. For all $\mathcal{V}_1, \mathcal{V}_2 \in \mathbb{V}$ with representation maps $m_1 \in \mathcal{V}_1 \rightarrow \mathcal{N}, m_2 \in \mathcal{V}_2 \rightarrow \mathcal{N}$, and for all $n_1 \in \mathcal{V}_1$ with locally sound abstraction $s_1^\# \in \Sigma^\#$ and $n_2 \in \mathcal{V}_2$ with locally sound abstraction $s_2^\# \in \Sigma^\#$ (both for some $W \in \mathcal{W}_P$), if $[m_1(n_1)] = [m_2(n_2)]$, then $s_1^\# \sqcap s_2^\#$ is a locally sound abstraction for W of n_1 and n_2 .

PROOF. Since $[m_1(n_1)] = [m_2(n_2)]$, $\llbracket n_1 \rrbracket(W) = \llbracket n_2 \rrbracket(W)$ by Definition 5.1. By assumption, $\llbracket n_1 \rrbracket(W) \sqsubseteq \gamma(s_1^\#)(W)$ and $\llbracket n_2 \rrbracket(W) \sqsubseteq \gamma(s_2^\#)(W)$. Since $\llbracket n_1 \rrbracket(W) = \llbracket n_2 \rrbracket(W)$, $\llbracket n_1 \rrbracket(W) \sqsubseteq \gamma(s_2^\#)(W)$. Thus, $\llbracket n_1 \rrbracket(W) \sqsubseteq \gamma(s_1^\#)(W) \sqcap \gamma(s_2^\#)(W) \sqsubseteq \gamma(s_1^\# \sqcap s_2^\#)(W)$, so $s_1^\# \sqcap s_2^\#$ is a locally sound abstraction for W of n_1 . The same argument applies for n_2 . $\square$

We can perform e-class analysis by analyzing the e-nodes in each e-class and combining the abstractions with the abstract meet operator to get a single “summary” abstraction for the e-class.

As in prior work, we can compute e-class analysis as a (post-)fixpoint of a set of equations [13].

Definition 5.4 (Equations for E-Class Analysis of SSA Programs). Fix a program with sound e-graph $(\mathcal{N}, C)$. Its e-class analysis equations are given by:

⁸Technically, when combined with an e-graph, $\mathcal{G}$ must be modified to use e-classes as guards rather than nodes in a cyclic term. Then, when considering the program generated by considering a particular represented graph, $\mathcal{G}$ must have each guard e-class substituted with a value from the represented graph that maps into that e-class.

- For each e-class $c \in C$, the equation is $c^\# = \bigcap_{n \in [c]^{-1}} n^\#$.
- For each e-node $n \in N$, the equation is the same as described in Definition 4.6.

In practice, the equations for e-nodes are inlined into the equations for e-classes, as in Figure 3b.

The reason that e-class analyses can be more precise than standard abstract interpretation on individual represented graphs of the e-graph is because abstract interpretation is inherently *intensional*—that is, the way a program is written can affect the computed analysis result, even if the extensional behavior of the program does not change [4, 19]. Performing rewriting in an e-graph with equality saturation exposes equivalences between multiple different ways of writing the same program. These syntactically different but semantically equivalent programs may be analyzed differently and e-class analysis allows us to combine these results to increase precision. We describe a specific example of this phenomenon in Section 2.1.3.

Unfortunately, some (post-)fixpoints of the standard equations for e-class analysis, as given in Definition 5.4, will compute *unsound* e-class analyses for some programs. This was demonstrated with particular examples in Section 3. The core issue is that while a sound e-class analysis is an over-approximation of the semantics of *well-formed* represented graphs, whether a represented graph is well-formed or ill-formed is a subtle property that is not immediately evident from the structure of the graph—the equations given in Definition 5.4 are produced by the immediate structure of an e-graph, and will thus “consider” *all* represented graphs. Since e-class analysis combines analysis results with the meet operator, (post-)fixpoints of the resulting equations will not necessarily over-approximate the semantics of well-formed represented graphs of the e-graph.

5.3 Computing a Sound and Optimistic E-Class Analysis

We propose an algorithm that computes an e-class analysis of a SSA program which is 1) sound (according to Definition 5.2) and 2) optimistic (is able to analyze loops with some precision). Our algorithm relies on two observations regarding e-graphs and their represented graphs:

- (1) All cycles in an e-graph are either 1) cycles containing ϕ nodes that correspond to control flow loops or 2) cycles created by rewrites that equate a term to one of its sub-terms. We will call the first kind of cycle “well-formed” and the second kind “ill-formed”.
- (2) If the initial SSA program was well-formed, then after rewriting the set of ill-formed represented graphs is exactly the set of represented graphs containing ill-formed cycles.

Recall from Section 3 that a greatest fixpoint solution to the e-class analysis equations is actually a sound e-class analysis because no optimistic assumptions are made (and thus the computation is incrementally sound). Intuitively, our algorithm will allow optimistic assumptions for well-formed cycles while not allowing optimistic assumptions for ill-formed cycles.

Our algorithm proceeds as follows. First, identify the well-formed cycles in the e-graph. We accomplish this in the same way that widening points are determined. We compute a WTO over the CFG, identify component head vertices, and mark all control flow edges to component heads from a vertex in the same component as “back” edges. Well-formed cycles in the e-graph contain ϕ nodes whose vertex is a component head, and thus have an input corresponding to a back edge.

Second, compute an e-class analysis that is locally sound for all walks that *do not traverse any back edges*. For any ϕ with an input corresponding to a back edge, that input to the ϕ can effectively be ignored, since no walk being considered can possibly walk that edge. This breaks all well-formed cycles in the e-graph. The resulting e-graph can be analyzed using standard e-class analysis—specifically, we compute a greatest fixpoint to the e-class analysis equations, since this result is a (locally) sound e-class analysis. Call this result $S_0^\#$.

Third, compute an e-class analysis that is locally sound for all walks that *traverse back edges at most j times*, given a locally sound analysis for all walks that traverse back edges at most $j - 1$

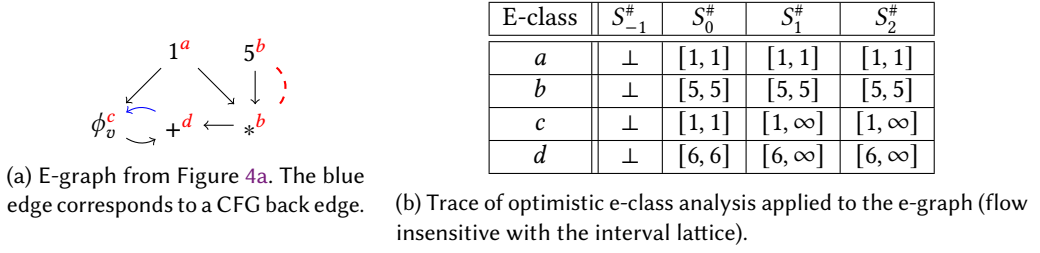


Fig. 5. A trace of optimistic e-class analysis applied to the program from Figure 2.

times. For any ϕ with an input corresponding to a back edge, that input to the ϕ can directly look up an analysis result from $S_{j-1}^\#$, rather than the result for that e-class currently being computed. This breaks all well-formed cycles in the e-graph, so we can compute a greatest fixpoint to these e-class analysis equations as well. Call this result $S_j^\#$.

Fourth, repeat the third step l total times to compute $S_l^\#$. Once $S_l^\# = S_{l+1}^\#$, return $S_l^\#$ as the result.

Definition 5.5 (Optimistic E-Class Analysis). Given a SSA program with sound e-graph $(\mathcal{N}, \mathcal{C})$ and CFG $\mathcal{G} = (V, s, P)$ and a set of back edges $B \in V \rightarrow \{1, 2\}$ (partial function from vertex to which incoming edge is a back edge) such that every cycle in $\mathcal{G}$ contains an edge from B ...

- $\mathcal{F}_j \in \mathcal{N} \rightarrow \Sigma^\#$ is the abstract transformer for e-nodes during the j th iteration.
- $S_j^\# \in \mathcal{C} \rightarrow \Sigma^\#$ is the intermediate e-class analysis result during the j th iteration.
- $\mathcal{F}_{-1} = [n \in \mathcal{N} \mapsto \perp]$.
- $\mathcal{F}_j(n) \triangleq \begin{cases} n_1^\# \sqcup n_2^\# & n \text{ is a } \phi_v \wedge v \notin \text{dom}(B) \\ \mathcal{F}_{j-1}(n) \nabla (S_{j-1}^\#(n_1) \sqcup n_2^\#) & n \text{ is a } \phi_v \wedge B(v) = 1 \\ \mathcal{F}_{j-1}(n) \nabla (n_1^\# \sqcup S_{j-1}^\#(n_2)) & n \text{ is a } \phi_v \wedge B(v) = 2 \\ f^\#(n_1^\#, \dots, n_k^\#) & n \text{ is a } f \in \Sigma^k \rightarrow \Sigma \end{cases}, \text{ for } j \geq 0.$
- $S_{-1}^\# \triangleq [c \in \mathcal{C} \mapsto \perp]$.
- $S_j^\# \triangleq S_{j-1}^\# \sqcup \text{gfp}(\{c^\# = \bigcap_{n \in [c]^{-1}} \mathcal{F}_j(n) \mid c \in \mathcal{C}\})$, for $j \geq 0$ ⁹.
- The final optimistic e-class analysis result is $S_l^\#$, where $S_l^\# = S_{l+1}^\#$.

THEOREM 5.6. Given a SSA program with sound e-graph $(\mathcal{N}, \mathcal{C})$ and CFG $\mathcal{G} = (V, s, P)$, back edges $B \in V \rightarrow \{1, 2\}$, and some l where $S_l^\# = S_{l+1}^\#$, then $S_l^\#$ is a sound e-class analysis of the program.

PROOF. Fix a well-formed represented graph of the e-graph, called $\mathcal{V}$, and its representation map, $m \in \mathcal{V} \rightarrow \mathcal{N}$. We will show that $S_l^\#$ is a sound abstraction of the SSA program $\mathcal{V}, \mathcal{G}$.

First, we show that $S_0^\#$ is a locally sound abstraction for all possible walks that do not traverse an edge in B . Fix such a walk W . $S_0^\#$ is at least the greatest fixpoint of a set of equations—we proceed by showing that 1) the initial assignments of $c^\#$ for $c \in \mathcal{C}$ are locally sound and 2) for each equation, if the previous values of $c^\#$ for $c \in \mathcal{C}$ are locally sound, then the new value for some $c^\#$ computed by the equation is still locally sound.

The initial assignments for each e-class is $\top^\#$, since we are computing a greatest fixpoint. $\forall s \in \Sigma, s \sqsubseteq \top$, so $\forall n \in \mathcal{V}, \llbracket n \rrbracket(W) \sqsubseteq \top = \gamma(\top^\#)$.

Fix a node $n \in \mathcal{V}$, its e-node $n' = m(n)$, and its e-class $c = [n']$. If n is a $f \in \Sigma^k \rightarrow \Sigma$ node with sound abstract transformer $f^\#$, then $\mathcal{F}_0(n') = f^\#(n_1^\#, \dots, n_k^\#)$ is a sound abstraction of n . If n is a ϕ_v node for some $v \in V$, there are three cases. In the first, neither input to the ϕ corresponds to

⁹The join with $S_{j-1}^\#$ ensures that $S^\#$ is non-decreasing if ∇ is not monotone. The join can be dropped if ∇ is monotone.

an edge in B ($v \notin \text{dom}(B)$). Thus, W may traverse either predecessor of v . $\mathcal{F}_0(n') = n_1^{\#} \sqcup n_2^{\#}$ —the abstract lattice join operator over-approximates ϕ nodes, so this is an over-approximation of n . In the second and third cases, one input to the phi corresponds to an edge in B —without loss of generality, let us say $B(v) = 1$. $\mathcal{F}_0(n') = \mathcal{F}_{-1}(n') \nabla (S_{-1}^{\#}(n_1') \sqcup n_2^{\#}) \supseteq n_2^{\#}$. This over-approximates n , since $\llbracket n \rrbracket(W)$ is either 1) $\perp$ if W has not traversed v or 2) $\llbracket n_2 \rrbracket(W')$ if W has traversed v , both of which are over-approximated by $n_2^{\#}$. $\mathcal{F}_0(n')$ is a locally sound abstraction of n in all cases—by Theorem 5.3, $\bigcap_{n'' \in [c]^{-1}} \mathcal{F}_0(n'')$ is a locally sound abstraction of n as well. This is the equation for the abstraction of the e-class c , so the equation preserves local soundness.

Second, we show that if $S_j^{\#}$ is a locally sound abstraction for all possible walks that traverse edges from B at most j times, then $S_{j+1}^{\#}$ is a locally sound abstraction for all possible walks that traverse edges from B at most $j + 1$ times. Fix a possible walk W that traverses edges from B at most $j + 1$ times. We proceed similarly as in the $S_0^{\#}$ case. $S_{j+1}^{\#}$ is at least a particular greatest fixpoint—we have already shown that $[c \in C \mapsto \top^{\#}]$ is locally sound.

Fix a node $n \in \mathcal{V}$, its e-node $n' = m(n)$, and its e-class $c = [n']$. If n is a $f \in \Sigma^k \rightarrow \Sigma$ node with sound abstract transformer $f^{\#}$, then $\mathcal{F}_{j+1}(n')$ is a locally sound abstraction of n . If n is a ϕ_v node for some $v \in V$, there are three cases. In the first case, neither input to the ϕ corresponds to an edge in B . This case is shown in the same way as for $S_0^{\#}$. In the second and third cases, one input to the phi corresponds to an edge in B —without loss of generality, let us say $B(v) = 1$. $\mathcal{F}_{j+1}(n') = \mathcal{F}_j(n') \nabla (S_j^{\#}(n_1') \sqcup n_2^{\#}) \supseteq S_j^{\#}(n_1') \sqcup n_2^{\#}$. Assume that W ends in v (if it does not, then either $\llbracket n \rrbracket(W) = \llbracket n \rrbracket(W')$, so consider the predecessor walk W' instead, or $W = (s)$ so $\llbracket n \rrbracket(W) = \perp$, which is over-approximated by any abstraction). If this walk traversed the first predecessor of v , then the predecessor walk W' traversed edges from B at most j times—this is because the edge from the first predecessor of v to v is a back edge. Thus, $\llbracket n \rrbracket(W) = \llbracket n_1 \rrbracket(W')$, for which $S_j^{\#}(n_1')$, $S_j^{\#}(n_1') \sqcup n_2^{\#}$, and $\mathcal{F}_{j+1}(n')$ are locally sound abstractions. If W traversed the second predecessor of v , then $\llbracket n \rrbracket(W) = \llbracket n_2 \rrbracket(W')$, for which $n_2^{\#}$, $S_j^{\#}(n_1') \sqcup n_2^{\#}$, and $\mathcal{F}_{j+1}(n')$ are locally sound abstractions. $\mathcal{F}_{j+1}(n')$ is a locally sound abstraction of n in all cases—by Theorem 5.3, $\bigcap_{n'' \in [c]^{-1}} \mathcal{F}_{j+1}(n'')$ is a locally sound abstraction of n as well. This is the equation for the abstraction of the e-class c , so the equation preserves local soundness.

Every walk is finite, so every walk visits back edges a finite number of times. If a possible walk visits back edges at most l times, then $S_l^{\#}$ is a locally sound e-class analysis. Consider a possible walk that visits back edges at most l' times, for some $l' > l$. Since $S_l^{\#} = S_{l+1}^{\#}$, and every $S_j^{\#}$ is defined directly in terms of $S_{j-1}^{\#}$, it must be the case that $S_l^{\#} = S_{l+1}^{\#} = S_{l+2}^{\#} = \dots = S_{l'}^{\#}$, so $S_l^{\#}$ is still a locally sound e-class analysis. Thus, $S_l^{\#}$ is a locally sound e-class analysis for all possible walks. $\square$

5.4 Precision and Complexity

Notably, the proof of correctness given in Section 5.3 assumes very little of the set of back edges¹⁰. However, it is advantageous to pick this set carefully for precision and complexity reasons. If this set is too small, then some loops in the original program will not have an edge represented—these loops will not be analyzed optimistically. If this set is too large, then there will be “too many” widening points. It is well known that minimizing the set of points at which widening occurs is advantageous for precision [3]. Figure 5 shows an example trace of optimistic e-class analysis being applied to the running example program—the result is more precise than standard e-class analysis.

Given $S_j^{\#}$, the complexity to compute $S_{j+1}^{\#}$ scales with the number of e-nodes, since this bounds the number of equations. As in prior work on e-class analysis, we use a worklist algorithm that propagates the analysis equations for each e-node one-at-a-time [56]. The maximum amount of

¹⁰The only assumption is that B is a partial function, so there is no $v \in V$ such that $(v, 1) \in B \wedge (v, 2) \in B$.

times any single equation can be propagated is the maximum length of a descending chain in the abstract lattice. This length is theoretically large, except 1) since this is an incrementally (locally) sound analysis, the length can be cut off with narrowing and 2) we show in Section 7.3 that the average number of times equations must be propagated before a fixpoint is reached is very small.

The quantity l for which $S_l^\# = S_{l+1}^\#$ is bounded by the maximum length of an ascending chain in the abstract lattice (shortened with widening) times $|\{n \in \mathcal{N} \mid n \text{ is a } \phi_v \wedge v \in \text{dom}(B)\}|$. This is the same bound as the maximum number of iterations over a connected component of the “iterative” chaotic iteration strategy that is commonly used in the abstract interpretation literature [3]. We also show that in practice $S_l^\# = S_{l+1}^\#$ for a very small l in Section 7.3.

Call D the maximum length of a descending chain (possibly with narrowing), A the maximum length of an ascending chain (possibly with widening), and $L = |\{n \in \mathcal{N} \mid n \text{ is a } \phi_v \wedge v \in \text{dom}(B)\}|$. The complexity of optimistic e-class analysis is $O(|\mathcal{N}| * D * A * L)$, though we show in Section 7.3 that this is a very conservative bound.

6 Optimism in Equality Saturation

We now consider how we can combine optimistic e-class analysis with equality saturation. We propose an algorithm that alternates between phases of optimistic e-class analysis and equality saturation, using the improved precision in one half to improve the precision of the other half.

6.1 Using Analysis Results in Equality Saturation

First, we discuss how analysis results can help equality saturation. Recall from Section 2.1 that equality saturation consists of the repeated application of *rewrite rules* to an e-graph. A rewrite rule performs *e-matching* to find represented terms matching some pattern—for each matched term, an *action* is taken, usually to insert a new represented term and assert that sets of e-classes are now known equal. A rewrite rule may additionally depend on a set of *conditions*—that is, even if the pattern e-matches some represented term, some analysis fact must also be known about involved e-classes for the action to be sound. For example, in a setting involving bitvector operations, the rewrite $a/b \Rightarrow a \gg \log_2(b)$ is only valid if b is known to be a power of two [35].

To depend on $\Sigma^\#$ in rewrite rules, we require that $\Sigma^\#$ is a flow-insensitive abstraction. Rewrite rules equate e-classes *globally* and for the e-graph to be sound, it must be true that all e-nodes in the same e-class evaluate to the same value on all possible walks. Thus, a rewrite rule can only use an abstraction that is sound on all possible walks. We discuss a potential relaxation of this requirement in Section 6.3. Additionally, we require that if a rewrite rule fires given an e-class analysis $S^\#$, it must fire given an e-class analysis $S^{\#'}$ where $S^{\#'} \sqsubseteq S^\#$. This ensures that if analysis results improve, the set of applicable rewrites only grows.

6.2 Combined Optimistic E-Class Analysis and Equality Saturation

Standard e-class analysis can be run at the same time as rewrites and/or rebuilding, where the definition of saturation is extended to include that abstractions of e-classes stop changing [56, 60]. This is only true because standard e-class analysis is incrementally sound. This is not necessarily true in optimistic e-class analysis—for arbitrary j , $S_j^\#$ is not necessarily a sound e-class analysis. It is well known that optimistic analyses can be combined via the *reduced product* [9]. Unfortunately, existing versions of equality saturation are pessimistic: 1) it is not clear how a “ $\perp$ e-graph” could be stored, 2) rewrites are never thrown away and assert soundness as a pre-condition and post-condition, and 3) saturation is not guaranteed, so being able to stop early is essential in practice.

We propose that optimistic e-class analysis and equality saturation be run in two separate halves—these halves execute repeatedly and results are propagated between the halves. This ensures that

Algorithm 1 Optimistic e-class analysis and equality saturation combination

```

1: procedure OPTIMISMINEQUALITYSATURATION( $(N, C), \mathcal{G}$ )
2:    $B \leftarrow \text{WTOBACKEDGES}(\mathcal{G})$  //  $\mathcal{G}$  does not change during analysis or rewriting
3:    $S^\# \leftarrow [c \in C \mapsto \top]$ 
4:   while  $(N, C)$  has changed and before timeout do
5:      $S^\# \leftarrow S^\# \sqcap \text{OPTIMISTICECLASSANALYSIS}((N, C), B)$ 
6:     while  $(N, C)$  has changed and before timeout do
7:       Apply rewrites to  $(N, C)$  using  $S^\#$ 
8:       Perform rebuilding on  $(N, C)$ 
9:     end while
10:  end while
11:  return  $(N, C), S^\#$ 
12: end procedure

```

only sound results derived from the e-class analysis are used for rewrites. Algorithm 1 shows pseudo-code for this approach, which we call “optimism in equality saturation”.

Due to aforementioned restrictions on rewrite rules, more precise abstractions will never result in an e-graph with fewer rewrites applied. However, an e-graph with more applied rewrites will not necessarily result in at least as precise an analysis, because widening is not necessarily monotone. Since every intermediate e-graph is sound, any intermediate analysis result is also sound—therefore, the intermediate analysis $S^\#$ can be forced to never increase by meeting it with the previous iteration’s result (line 5). With this modification, the precision of both the rewriting and analysis halves either improve each iteration or do not change. Every intermediate step of this algorithm stores 1) a sound e-graph and 2) a sound analysis result. Therefore, both loops may be run on a timeout or a maximum number of iterations, as is common in equality saturation applications.

6.3 Using Flow Sensitive Abstractions in Contextual E-Graphs

To use flow sensitive abstractions in rewrites, one would need to be able to record equivalences in a flow sensitive manner. However, the equivalence relation that an e-graph stores is flow insensitive (the equivalences between nodes must hold on *all* possible walks). Conceptually, we can modify the equivalence relation to be flow sensitive. Then, when a rewrite rule depends on a flow sensitive abstraction, any discovered equalities are recorded flow sensitively. In prior work, e-graphs that can store multiple equivalence relations efficiently have been called “contextual e-graphs” [17, 21] or “colored e-graphs” [47, 48]. Equivalence relations are identified by either “contexts” or “colors” and form a hierarchy or lattice relationship. Intuitively, if a context A is the parent of another context B in the hierarchy, then any equalities known in A are automatically known in B . For implementing a flow sensitive equivalence relation with a contextual e-graph, we suggest that dominance in a CFG is a good choice of hierarchy. Implementing contextual e-graphs efficiently is an open research question, and we hope this perspective serves as a motivating use case for future work.

7 Examples and Evaluation

In this section, we describe how optimism in equality saturation applies to a set of example programs. We implement optimistic e-class analysis and equality saturation in a small Rust tool that correctly analyses the example programs. We also evaluate this tool on a set of randomly generated programs to 1) confirm that the rewritten e-graphs and e-class analyses are indeed sound and 2) evaluate the performance of optimistic e-class analysis, compared with standard abstract interpretation and standard e-class analysis. All reported wall clock times constitute an average of 25 runs. All experiments were run on a standard laptop (Intel Core Ultra 7 155H @ 1.4 GHz w/ 32GB LPDDR5

memory). The code for the implementation, its test suite, and its empirical evaluation are all available in the software artifact accompanying this paper [2].

7.1 Concrete Implementation

We implemented optimism in equality saturation for SSA programs in a small Rust program. The tool parses programs in a pseudo-code syntax and translates each function into a SSA program. Optimistic e-class analysis and equality saturation are run as described in Section 6.2.

We implement three analyses—intervals, global value numbering (GVN), and reachability. Interval analysis is the standard non-relational abstraction of numeric values consisting of a range from a low integer (or $-\infty$) to a high integer (or ∞) [10, 11]. GVN analysis computes a value number per e-node in the e-graph—a conditional rewrite merges e-nodes whose value numbers are the same. $\perp_{\text{GVN}}$ is a placeholder that is equal to itself, so back edge inputs tophis are optimistically assumed to be equal—this is what allows us to de-duplicate isomorphic well-formed cycles. $\top_{\text{GVN}}$ is a placeholder that is considered not equal to itself, so the greatest fixpoint computation during optimistic e-class analysis is incrementally locally sound. Reachability analysis abstracts what control flow walks are possible. We split reachability analysis into two cooperating abstractions—vertex reachability and edge reachability. As reachability analysis characterizes the CFG rather than the SSA e-graph, we implement it as a separate cooperating analysis in a reduced product with the optimistic e-class analysis. The reachability abstraction makes the interval and GVN abstractions more precise by eliminating unreachable edge inputs to ϕ nodes. The interval abstraction makes the reachability abstraction more precise by identifying when an edge’s condition will never be met (in our implementation, an edge will never be met if the interval $[0, 0]$ can be derived for the condition value)—this effectively captures sparse conditional constant propagation [55].

Notably, interval domains have either long descending chains or infinite descending chains, depending on the exact implementation. This violates an assumption (given in Definition 4.5) we use for our formal description of optimistic e-class analysis. In a concrete implementation, we can compute a *bounded* sequence of incrementally locally sound abstractions, rather than a greatest fixpoint, to maintain soundness while allowing our algorithm to terminate. This matches observations made in prior work regarding the termination of e-class analysis [13]. In practice, our tool is always able to compute greatest fixpoints for every input program used in our evaluation.

7.2 Example Programs

The first example program is shown in Figure 6. The “goal” is to discover that the return value is equal to 49. At a high level, discovering this fact requires an optimistic interval analysis to discover that $x = 2$ at line 7¹¹, rewrites to discover that $((2 + y) + z) * y = 2 * y + (y * y + z * y)$, a rewrite to discover that $(2 * y + (y * y + z * y)) \neq 2 * y + (y * y + z * y) = 0$, and finally a reachability analysis to discover that line 10 is unreachable.

The second example program is shown in Figure 7. The “goal” is to discover that the return value is equal to 0. Since x is a function parameter, its interval is $[-\infty, \infty]$ —to discover $x = y$, the GVN analysis must find that the SSA values being subtracted at the end of the function have the same value number. Rewriting discovers that $xt * (y + 5 + 0) = xt * y + xt * 5$. Then, the optimistic GVN analysis can derive that x and y share a value number, and a rewrite merges their e-classes. The final return constant value is discovered by the rewrite $x - x \Rightarrow 0$. Note that we include the rewrite rule $x + 0 \Rightarrow x$ in our rule set, so we discover an ill-formed cycle due to the sub-expression $5 + 0$. This ill-formed cycle does not poison our optimistic e-class analysis.

¹¹We define flow insensitive abstractions over SSA values, not program variables, so our tool can find abstractions of x at different locations when x corresponds to different values at those locations.

```

1 fn example1(y) {
2   let x = -6;
3   let z = 42;
4   while y < 10 {
5     y = y + 1;
6     x = x + 8;
7     let lhs = ((x + y) + z) * y;
8     let rhs = 2 * y + (y * y + z * y);
9     if lhs != rhs {
10      z = 24;
11    }
12    x = x - 8;
13  }
14  return z + 7;
15 }

```

Fig. 6. First example program to analyze. The goal is to show that the returned value is 49, which requires rewriting, reachability analysis, and interval analysis.

```

1 fn example2(x) {
2   let y = x;
3   while y < 10 {
4     let xt = x;
5     x = y * y + y * 5;
6     y = xt * (y + 5 + 0);
7   }
8   return x - y;
9 }

```

Fig. 7. Second example program to analyze. The goal is to show that the returned value is 0, which requires rewriting and GVN analysis.

The second example demonstrates a capability of optimistic e-class analysis that has eluded prior equality saturation systems: soundly de-duplicating cycles. Our rewrite set includes a rule that merges the e-classes of two e-nodes if the GVN analysis derives the same value number for both e-nodes. Since the GVN analysis is optimistic, two well-formed cycles can be discovered to share value numbers (more precisely, there is a mapping between e-classes in each cycle such that each pair of e-classes in the mapping share value numbers). In effect, this allows us to merge well-formed cycles, which is sound, while not merging ill-formed cycles, which is unsound. This addresses a known problem in the e-graphs literature [51, 62, 63].

We wrote these examples both in a pseudo-code language for our tool (shown in Figures 6 and 7) and in C—we compiled the examples in C with GCC 15.2 and Clang 21.1.0 (with optimization level -O2). Neither GCC nor Clang can fully optimize either example. We ran our tool on both examples in three configurations: 1) standard abstract interpretation before rewriting, 2) standard e-class analysis during rewriting, and 3) optimistic e-class analysis during rewriting. Neither standard abstract interpretation nor standard e-class analysis derives the correct constant for the return value of each function, while optimism in equality saturation does.

7.3 Evaluation on Randomly Generated Programs

We randomly generated 100 programs of varying sizes to 1) confirm that optimism in equality saturation can soundly analyze and rewrite programs and 2) empirically evaluate the performance of optimistic e-class analysis. After rewriting, the largest e-graph contained 11375 e-nodes and the most loops in a single program was 184.

First, we ran each program in a reference interpreter and checked if optimistic e-class analysis computed over-approximations of the programs' concrete behaviors (before and after rewriting). The analyses were sound for all generated programs (this should not be surprising—we proved that optimistic e-class analysis computes a sound abstraction of well-formed represented graphs).

Second, we ran optimistic e-class analysis on each program after several rounds of rewriting and measured 1) the number of iterations of the greatest fixpoint computation to compute each $S_l^\#$ and 2) for what l is $S_l^\# = S_{l+1}^\#$. In practice, the greatest fixpoint visits e-nodes, rather than e-classes,

Table 1. Wall clock time (in μs) for standard abstract interpretation, standard e-class analysis, and optimistic e-class analysis run on 100 programs, along with the # of visit items and the # of fixpoint computations (n). The relative execution time for optimistic e-class analysis is shown in parentheses compared to standard abstract interpretation and standard e-class analysis. We compute the arithmetic mean of absolute measurements and the geometric mean of relative measurements. The # of visit items after rewriting is shown relative to before. Standard abstract interpretation is run before rewriting while both e-class analyses are run after.

	Min	Median	Mean	Max
Standard Abstract Interpretation (μs)	5.87	252.95	755.35	7244.56
Standard E-Class Analysis (μs)	3.61	160.87	478.59	2320.57
Optimistic E-Class Analysis (μs)	6.16 (0.95x, 1.57x)	448.64 (1.44x, 3.01x)	1480.11 (1.88x, 2.94x)	7723.35 (34.20x, 5.28x)
# Visit Items Pre-rewriting	47.00	721.00	1405.57	9271.00
# Visit Items Post-rewriting	50.00 (0.98x)	1222.00 (1.35x)	3034.69 (1.78x)	12993.00 (19.75x)
n for Standard Abstract Interpretation	2.00	3.00	2.95	6.00
n for Optimistic E-Class Analysis	2.00	3.00	2.95	6.00

and also visits vertices and edges in the CFG (to propagate the reachability analysis). We call each of these objects “visit items”¹². We measured, for each program, the average number of times the greatest fixpoint computation visits each visit item. The average of this average across all generated programs was 3.19 and the maximum of this average was 4.38. In other words, the greatest fixpoint computation visits each visit item a small number of times on average. If for a fixed program we have a minimum l such that $S_l^\# = S_{l+1}^\#$, then optimistic e-class analysis performs $n = l + 2$ greatest fixpoint computations. We measure n for each generated program—the maximum observed n was 6 and for 99% of programs n was 3 or less. Note that the theoretical bound on n is a multiple of the number of ϕ e-nodes whose vertex is a component head, which at its largest was 3068.

Third, we measured the execution time of standard abstract interpretation (using iterative chaotic iteration), standard e-class analysis, and optimistic e-class analysis on the generated programs. Our results are summarized in Table 1. We observed that in the mean and median cases, the slowdown from using optimistic e-class analysis over standard abstract interpretation was similar to the factor increase in the number of visit items after performing rewriting ($1.44 \approx 1.35$ and $1.88 \approx 1.78$). Additionally, the slowdown over standard e-class analysis was similar to the factor n that corresponds to computing a greatest fixpoint multiple times ($3.01 \approx 3$, $2.94 \approx 2.95$). In other words, the slowdown in using optimistic e-class analysis is associated with, depending on the baseline, either 1) the increase in the size of the e-graph from rewriting or 2) from iteratively computing an abstraction in a similar fashion as the standard iterative chaotic iteration strategy.

8 Related Work

E-Graphs and Equality Saturation. E-graphs were originally developed for computing congruence closure over a set of terms [37] and are a key ingredient in SMT solvers for propagating equalities between theories [38]. Later work proposed applying rewrite rules to e-graphs via e-matching during equality saturation [49, 51, 56]. Most e-matching implementations are based on pattern compilation [34] or relational queries [61]. Another consideration is the representation of the

¹²The number of visit items is the number of e-nodes plus the number of CFG vertices plus the number of CFG edges.

e-graph—traditionally, e-graphs are stored directly as graphs in memory [15, 56], while recent work proposes using database tables with a canonicalization procedure [59, 60]. Acyclic term languages are the most common target for equality saturation. Cycles are often created via rewrite rules, but are rarely desired and often filtered or ignored during extraction [5, 57, 58]. The only prior work that we are aware of that encodes an explicitly cyclic program representation into e-graphs is Peggy [51], which does not support lattice-based analyses¹³. Neither egg nor egglog¹⁴ support creating an initial e-graph with cycles and both tools treat extracting cycles as a failure mode [56, 60].

Abstract Interpretation over E-Graphs. E-class analysis is a technique for performing lattice-based analyses on e-graphs. A fact from a single lattice is assigned to each e-class in an e-graph [56]. Standard e-class analysis is computed bottom-up and all facts are conservatively initialized to $\top$. Facts are propagated during rewriting, since the analysis is incrementally sound. Later work formalizes e-class analysis as an abstract interpretation and observes the fruitful back-and-forth between analysis and rewriting, but does not produce a precise result for well-formed cycles in a cyclic program representation [13]. egglog allows users to define analyses with functional database tables. Unlike in egg, the database approach supports 1) storing multiple analyses per e-class and 2) relational analyses involving multiple e-classes. However, the restriction of incremental soundness still applies, which prevents analyzing cyclic programs optimistically. To our knowledge, no prior equality saturation system supports optimistic analyses of well-formed cycles.

Equivalence and Relational Abstractions. Several relational abstract domains have been proposed, including difference bounds [31], difference abstraction [32], octagons [33, 45], pentagons [30], polyhedra [12, 46], Karr’s domain [23], two variables per inequality [43], and equalities (using e-graphs to store the equivalence relation) [6]. E-class analysis has not yet been extended to arbitrary relational domains—we believe this is an important direction for future work. As relational abstractions store an abstract value per pair, or even tuple, of variables, the entire abstract state can grow quite large. Some prior work takes advantage of sparseness in the relations between variables [45]. Another approach uses a data structure called a “labeled union find” to store some weakly relational domains by a spanning tree, rather than a fully connected graph [29]. A weakly relational domain can be used to “map factorize” compatible non-relational domains, meaning the non-relational domain is only stored per relational class, rather than per program variable. The equivalence relation stored in an e-graph has been viewed by some prior work as an abstraction [6] and can be seen as a maximal version of map factorization—e-class analysis stores a fact per e-class, not per e-node, and is compatible with every non-relational abstraction. E-graphs not only factor abstractions by equivalence classes but also *terms*—this is how e-graphs compactly represent a large (sometimes infinite) set of equivalent terms [37]. An interesting direction for future work could be exploring factoring terms by relations other than equivalence relations—labeled union finds could be used to factor terms that are equivalent modulo a group action, for example [63].

Combining Analyses and Transformations. The structure of many compilers consists of phases of analyses and transformations—either the order of analyses and transformations is designed explicitly [39] or a generic “pass” mechanism provides flexible ordering [24, 25, 42]. Every intermediate program is well-formed and optimistic analyses are often employed. However, intermediate programs are ephemeral and transformations are run in a specific order, leading to the classic phase ordering problem. Program transformations have also been used specifically as an ad-hoc mechanism to facilitate communication between program analyses [27]. Another approach to combining analyses and transformations is to view certain transformations themselves as abstract

¹³Some simple analyses, such as constant propagation, are re-implemented as pure syntactic transformations in Peggy.

¹⁴egglog’s relational e-matching can theoretically match graph patterns, not just term patterns [61].

interpretations. Recent work has applied this perspective to SSA translation [26] and to simple program simplifications [28]. Formalizing transformations as abstract interpretations allows for straightforward combinations with standard analyses via product operators [28], unlike the combination we propose in Section 6.2. We believe a version of “optimistic rewriting” may be possible to arrive at by formalizing equality saturation as an abstract interpretation. However, we also believe that the implementation of such a method would be difficult to optimize—an efficient implementation of persistent or backtrack-able e-graphs is likely necessary.

9 Conclusion

This paper tackles the problem of performing optimistic analyses over e-graphs in tandem with equality saturation. We identify a key problem that can cause optimistic e-class analysis to compute unsound results over e-graphs—equality saturation can create ill-formed represented graphs which poison the analysis with unsoundness. We propose an algorithm to compute optimistic e-class analyses that are sound for all well-formed represented graphs. This technique allows for sound and optimistic analysis of e-graphs containing cyclic program representations during equality saturation. Additionally, optimistic e-class analysis has a similar performance profile as existing abstract interpretation and e-class analysis techniques, meaning it is practical to use.

The treatment in this paper is primarily theoretical and discovers a qualitative improvement in capability over prior equality saturation systems. Important future work includes exploring domains where this technique could uncover quantitative gains in precision. Optimizing SSA programs is certainly a candidate for this kind of investigation. However, we believe that reconsidering the program representation itself could be fruitful—SSA programs separate data flow and control flow, and only data flow is represented in the e-graph. We believe optimism in equality saturation is applicable to other cyclic program representations, such as program expression graphs. Additionally, while we tackle optimistic e-class analysis in this paper, we do not propose an optimistic alternative to equality saturation—such an alternative may possess theoretical and/or practical advantages.

Acknowledgments

We are very grateful for discussions with Tyler Hou, Samuel Coward, Cheng Zhang, Alexandra Silva, George Constantinides, and more broadly at the Dagstuhl Seminar 26022, “Program Optimization with E-Graphs”. We would also like to thank our reviewers for their valuable feedback. This work is supported by NSF grants IIS-1955488, IIS-2027575, DOE awards DE-SC0016260, AC02-05CH11231, and DARPA Agreement No. HR00112590131.

Data Availability Statement

The tool described in Section 7 is open source and freely available at <https://github.com/RArbore/pldi26-artifact>. An archived version of the tool along with its evaluation set-up is available as a Docker image hosted on Zenodo [2].

References

- [1] B. Alpern, M. N. Wegman, and F. K. Zadeck. 1988. Detecting equality of variables in programs. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) (POPL '88). Association for Computing Machinery, New York, NY, USA, 1–11. doi:10.1145/73560.73561
- [2] Russel Arbore, Alvin Cheung, and Max Willsey. 2026. *Artifact for "Optimism in Equality Saturation" (PLDI 2026)*. doi:10.5281/zenodo.19581777
- [3] François Bourdoncle. 1993. Efficient chaotic iteration strategies with widenings. In *Formal Methods in Programming and Their Applications*, Dines Bjørner, Manfred Broy, and Igor V. Pottosin (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 128–141.

- [4] Roberto Bruni, Roberto Giacobazzi, Roberta Gori, Isabel Garcia-Contreras, and Dusko Pavlovic. 2019. Abstract extensionality: on the properties of incomplete abstract interpretations. *Proc. ACM Program. Lang.* 4, POPL, Article 28 (Dec. 2019), 28 pages. doi:10.1145/3371096
- [5] Yaohui Cai, Kaixin Yang, Chenhui Deng, Cunxi Yu, and Zhiru Zhang. 2025. SmoothE: Differentiable E-Graph Extraction. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 1020–1034. doi:10.1145/3669940.3707262
- [6] Bor-Yuh Evan Chang and K. Rustan M. Leino. 2005. Abstract Interpretation with Alien Expressions and Heap Structures. In *Verification, Model Checking, and Abstract Interpretation*, Radhia Cousot (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 147–163.
- [7] Cliff Click and Keith D. Cooper. 1995. Combining analyses, combining optimizations. *ACM Trans. Program. Lang. Syst.* 17, 2 (March 1995), 181–196. doi:10.1145/201059.201061
- [8] Cliff Click and Michael Paleczny. 1995. A simple graph-based intermediate representation. In *Papers from the 1995 ACM SIGPLAN Workshop on Intermediate Representations* (San Francisco, California, USA) (IR '95). Association for Computing Machinery, New York, NY, USA, 35–49. doi:10.1145/202529.202534
- [9] Agostino Cortesi, Giulia Costantini, and Pietro Ferrara. 2013. A Survey on Product Operators in Abstract Interpretation. *Electronic Proceedings in Theoretical Computer Science* 129 (09 2013). doi:10.4204/EPTCS.129.19
- [10] Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Los Angeles, California) (POPL '77). Association for Computing Machinery, New York, NY, USA, 238–252. doi:10.1145/512950.512973
- [11] Patrick Cousot and Radhia Cousot. 1979. Systematic design of program analysis frameworks. In *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (San Antonio, Texas) (POPL '79). Association for Computing Machinery, New York, NY, USA, 269–282. doi:10.1145/567752.567778
- [12] Patrick Cousot and Nicolas Halbwachs. 1978. Automatic discovery of linear restraints among variables of a program. In *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Tucson, Arizona) (POPL '78). Association for Computing Machinery, New York, NY, USA, 84–96. doi:10.1145/512760.512770
- [13] Samuel Coward, George A. Constantinides, and Theo Drane. 2023. Combining E-Graphs with Abstract Interpretation. In *Proceedings of the 12th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis* (Orlando, FL, USA) (SOAP 2023). Association for Computing Machinery, New York, NY, USA, 1–7. doi:10.1145/3589250.3596144
- [14] Samuel Coward, Theo Drane, and George A. Constantinides. 2024. ROVER: RTL Optimization via Verified E-Graph Rewriting. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024), 1–1. doi:10.1109/TCAD.2024.3410154
- [15] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: an efficient SMT solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems* (Budapest, Hungary) (TACAS'08/ETAPS'08). Springer-Verlag, Berlin, Heidelberg, 337–340.
- [16] Delphine Demange, Yon Fernández de Retana, and David Pichardie. 2018. Semantic reasoning about the sea of nodes. In *Proceedings of the 27th International Conference on Compiler Construction* (Vienna, Austria) (CC '18). Association for Computing Machinery, New York, NY, USA, 163–173. doi:10.1145/3178372.3179503
- [17] Alexandre Drewery, Thomas Jensen, and David Pichardie. 2025. Contextual Equality Saturation. In *SAS 2025 - 32nd Static Analysis Symposium*. Singapore, Singapore, 1–26. https://inria.hal.science/hal-05226543
- [18] Chris Fallin. 2023. ægraphs: Acyclic E-graphs for Efficient Optimization in a Production Compiler. https://pldi23.sigplan.org/details/egraphs-2023-papers/2/-graphs-Acyclic-E-graphs-for-Efficient-Optimization-in-a-Production-Compiler
- [19] Roberto Giacobazzi and Francesco Ranzato. 2025. The Best of Abstract Interpretations. *Proc. ACM Program. Lang.* 9, POPL, Article 46 (Jan. 2025), 31 pages. doi:10.1145/3704882
- [20] Amir Kafshdar Goharshady, Chun Kit Lam, and Lionel Parreaux. 2024. Fast and Optimal Extraction for Sparse Equality Graphs. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 361 (Oct. 2024), 27 pages. doi:10.1145/3689801
- [21] Tyler Hou, Shadaj Laddad, and Joseph M. Hellerstein. 2025. Towards Relational Contextual Equality Saturation. arXiv:2507.11897 [cs.PL] https://arxiv.org/abs/2507.11897
- [22] John B. Kam and Jeffrey D. Ullman. 1977. Monotone data flow analysis frameworks. *Acta Inf.* 7, 3 (Sept. 1977), 305–317. doi:10.1007/BF00290339
- [23] Michael Karr. 1976. Affine relationships among variables of a program. *Acta Inf.* 6, 2 (June 1976), 133–151. doi:10.1007/BF00268497
- [24] Chris Lattner and Vikram Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization* (Palo Alto, California) (CGO '04). IEEE Computer Society, USA, 75.

- [25] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: scaling compiler infrastructure for domain specific computation. In *Proceedings of the 2021 IEEE/ACM International Symposium on Code Generation and Optimization* (Virtual Event, Republic of Korea) (CGO '21). IEEE Press, 2–14. doi:10.1109/CGO51591.2021.9370308
- [26] Matthieu Lemerre. 2023. SSA Translation Is an Abstract Interpretation. *Proc. ACM Program. Lang.* 7, POPL, Article 65 (Jan. 2023), 30 pages. doi:10.1145/3571258
- [27] Sorin Lerner, David Grove, and Craig Chambers. 2002. Composing dataflow analyses and transformations. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Portland, Oregon) (POPL '02). Association for Computing Machinery, New York, NY, USA, 270–282. doi:10.1145/503272.503298
- [28] Dorian Lesbre and Matthieu Lemerre. 2024. Compiling with Abstract Interpretation. *Proc. ACM Program. Lang.* 8, PLDI, Article 162 (June 2024), 26 pages. doi:10.1145/3656392
- [29] Dorian Lesbre, Matthieu Lemerre, Hichem Rami Ait-El-Hara, and François Bobot. 2025. Relational Abstractions Based on Labeled Union-Find. *Proc. ACM Program. Lang.* 9, PLDI, Article 195 (June 2025), 26 pages. doi:10.1145/3729298
- [30] Francesco Logozzo and Manuel Fähndrich. 2008. Pentagons: a weakly relational abstract domain for the efficient validation of array accesses. In *Proceedings of the 2008 ACM Symposium on Applied Computing* (Fortaleza, Ceara, Brazil) (SAC '08). Association for Computing Machinery, New York, NY, USA, 184–188. doi:10.1145/1363686.1363736
- [31] Antoine Miné. 2001. A New Numerical Abstract Domain Based on Difference-Bound Matrices. In *Proceedings of the Second Symposium on Programs as Data Objects* (PADO '01). Springer-Verlag, Berlin, Heidelberg, 155–172.
- [32] Antoine Miné. 2002. A Few Graph-Based Relational Numerical Abstract Domains. In *Proceedings of the 9th International Symposium on Static Analysis* (SAS '02). Springer-Verlag, Berlin, Heidelberg, 117–132.
- [33] Antoine Miné. 2006. The octagon abstract domain. *Higher Order Symbol. Comput.* 19, 1 (March 2006), 31–100. doi:10.1007/s10990-006-8609-1
- [34] Leonardo Moura and Nikolaj Bjørner. 2007. Efficient E-Matching for SMT Solvers. In *Proceedings of the 21st International Conference on Automated Deduction: Automated Deduction* (Bremen, Germany) (CADE-21). Springer-Verlag, Berlin, Heidelberg, 183–198. doi:10.1007/978-3-540-73595-3_13
- [35] Manasij Mukherjee and John Regehr. 2024. Hydra: Generalizing Peephole Optimizations with Program Synthesis. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 120 (April 2024), 29 pages. doi:10.1145/3649837
- [36] Chandrakana Nandi, Max Willsey, Adam Anderson, James R. Wilcox, Eva Darulova, Dan Grossman, and Zachary Tatlock. 2020. Synthesizing structured CAD models with equality saturation and inverse transformations. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) (PLDI 2020). Association for Computing Machinery, New York, NY, USA, 31–44. doi:10.1145/3385412.3386012
- [37] Charles Gregory Nelson. 1980. *Techniques for program verification*. Ph.D. Dissertation. Stanford, CA, USA. AAI8011683.
- [38] Greg Nelson and Derek C. Oppen. 1979. Simplification by Cooperating Decision Procedures. *ACM Trans. Program. Lang. Syst.* 1, 2 (Oct. 1979), 245–257. doi:10.1145/357073.357079
- [39] Michael Paleczny, Christopher Vick, and Cliff Click. 2001. The java hotspot™ server compiler. In *Proceedings of the 2001 Symposium on Java™ Virtual Machine Research and Technology Symposium - Volume 1* (Monterey, California) (JVM'01). USENIX Association, USA, 1.
- [40] Pavel Panchekha, Alex Sanchez-Stern, James R. Wilcox, and Zachary Tatlock. 2015. Automatically improving accuracy for floating point expressions. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Portland, OR, USA) (PLDI '15). Association for Computing Machinery, New York, NY, USA, 1–11. doi:10.1145/2737924.2737959
- [41] Fabrice Rastello. 2016. *SSA-based Compiler Design* (1st ed.). Springer Publishing Company, Incorporated.
- [42] Amit Sabne. 2020. XLA : Compiling Machine Learning for Peak Performance.
- [43] Axel Simon and Andy King. 2010. The two variable per inequality abstract domain. *Higher Order Symbol. Comput.* 23, 1 (March 2010), 87–143. doi:10.1007/s10990-010-9062-8
- [44] Taylor Simpson, Keith Cooper, and L. Simpson. 1997. SCC-based value numbering. (02 1997).
- [45] Gagandeep Singh, Markus Püschel, and Martin Vechev. 2015. Making numerical program analysis fast. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Portland, OR, USA) (PLDI '15). Association for Computing Machinery, New York, NY, USA, 303–313. doi:10.1145/2737924.2738000
- [46] Gagandeep Singh, Markus Püschel, and Martin Vechev. 2017. Fast polyhedra abstract domain. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages* (Paris, France) (POPL '17). Association for Computing Machinery, New York, NY, USA, 46–59. doi:10.1145/3009837.3009885
- [47] Eytan Singher and Shachar Itzhaky. 2023. Colored E-Graph: Equality Reasoning with Conditions. arXiv:2305.19203 [cs.PL] <https://arxiv.org/abs/2305.19203>
- [48] Eytan Singher and Shachar Itzhaky. 2024. Easter Egg: Equality Reasoning Based on E-Graphs with Multiple Assumptions. In *2024 Formal Methods in Computer-Aided Design (FMCAD)*. 70–83. doi:10.34727/2024/isbn.978-3-85448-065-5_13

- [49] Michael Stepp, Ross Tate, and Sorin Lerner. 2011. Equality-based translation validator for LLVM. In *Proceedings of the 23rd International Conference on Computer Aided Verification (Snowbird, UT) (CAV'11)*. Springer-Verlag, Berlin, Heidelberg, 737–742.
- [50] Dan Suciu, Yisu Remy Wang, and Yihong Zhang. 2025. Semantic Foundations of Equality Saturation. In *28th International Conference on Database Theory (ICDT 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 328)*, Sudeepa Roy and Ahmet Kara (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 11:1–11:18. doi:10.4230/LIPIcs.ICDT.2025.11
- [51] Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. Equality saturation: a new approach to optimization. *SIGPLAN Not.* 44, 1 (Jan. 2009), 264–276. doi:10.1145/1594834.1480915
- [52] Todd Veldhuizen and Jeremy Siek. 2003. On Combining Program Improvers. (04 2003).
- [53] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2022. Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 254–265. doi:10.1109/CGO53902.2022.9741267
- [54] Yisu Remy Wang, Shana Hutchison, Jonathan Leang, Bill Howe, and Dan Suciu. 2020. SPORES: sum-product optimization via relational equality saturation for large scale linear algebra. *Proc. VLDB Endow.* 13, 12 (July 2020), 1919–1932. doi:10.14778/3407790.3407799
- [55] Mark N. Wegman and F. Kenneth Zadeck. 1991. Constant propagation with conditional branches. *ACM Trans. Program. Lang. Syst.* 13, 2 (April 1991), 181–210. doi:10.1145/103135.103136
- [56] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and extensible equality saturation. *Proc. ACM Program. Lang.* 5, POPL, Article 23 (Jan. 2021), 29 pages. doi:10.1145/3434304
- [57] Yichen Yang, Phitchaya Mangpo Phothilimtha, Yisu Remy Wang, Max Willsey, Sudip Roy, and Jacques Pienaar. 2021. Equality Saturation for Tensor Graph Superoptimization. In *Proceedings of Machine Learning and Systems*. arXiv:2101.01332
- [58] Jiaqi Yin, Zhan Song, Chen Chen, Yaohui Cai, Zhiru Zhang, and Cunxi Yu. 2025. e-boost: Boosted E-Graph Extraction with Adaptive Heuristics and Exact Solving. *arXiv preprint arXiv:2508.13020* (2025).
- [59] Yihong Zhang. 2022. PLDI: U: Towards a Relational E-graph. <https://api.semanticscholar.org/CorpusID:250122313>
- [60] Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. 2023. Better Together: Unifying Datalog and Equality Saturation. *Proc. ACM Program. Lang.* 7, PLDI, Article 125 (June 2023), 25 pages. doi:10.1145/3591239
- [61] Yihong Zhang, Yisu Remy Wang, Max Willsey, and Zachary Tatlock. 2022. Relational e-matching. *Proc. ACM Program. Lang.* 6, POPL, Article 35 (Jan. 2022), 22 pages. doi:10.1145/3498696
- [62] Philip Zucker. 2024. Co-Egraphs: Streams, Unification, PEGs, Rational Lambdas. <https://www.philipzucker.com/coegraph/>
- [63] Philip Zucker. 2025. Omelets Need Onions: E-graphs Modulo Theories via Bottom-up E-matching. arXiv:2504.14340 [cs.PL] <https://arxiv.org/abs/2504.14340>

Received 2025-11-13; accepted 2026-04-03